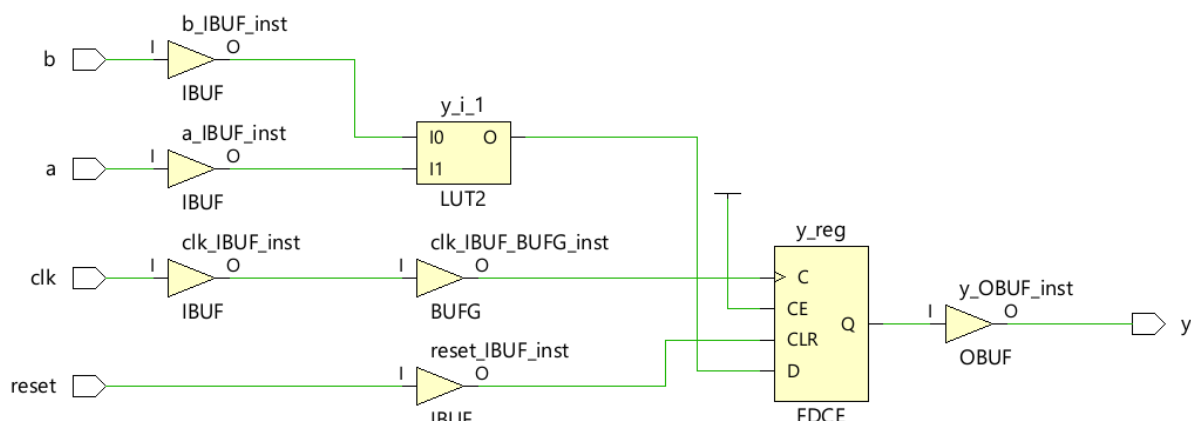


Assignment 3 - Common VHDL pitfalls and misconceptions

Task 1

Synthesize the design and look at the generated circuit (technology view.)



Inferred latches are often unintentional and generated from VHDL processes that were supposed to be purely combinatorial. Can you give an example?

If we only have a if-statement for some cases, but not all, memory will be needed. This is done by inferring a latch to retain the previous value.

How do you make sure that latches will not be generated from a combinatorial VHDL process?

Make sure every signal assigned should also receive a value on every possible execution path.

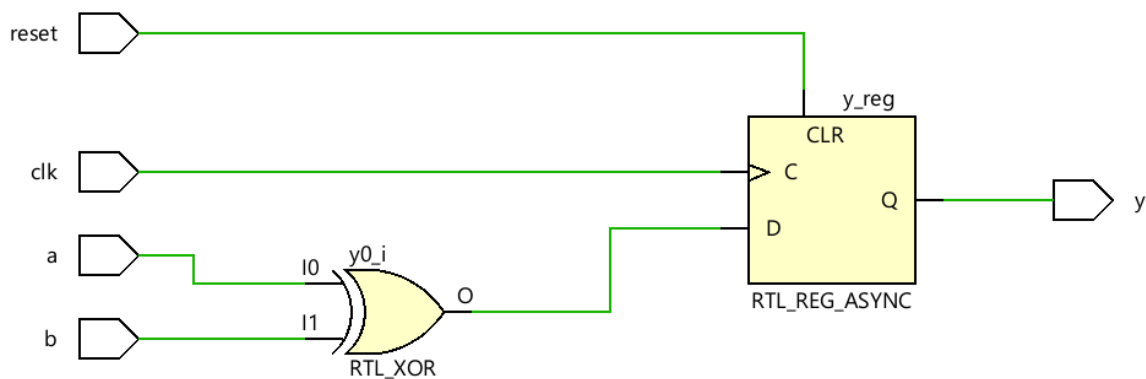
Task 2

The two simple designs below are both perfectly valid, but only one will implement the variable ' t ' as a register. Which one? Explain why this is so and how the two designs differ.

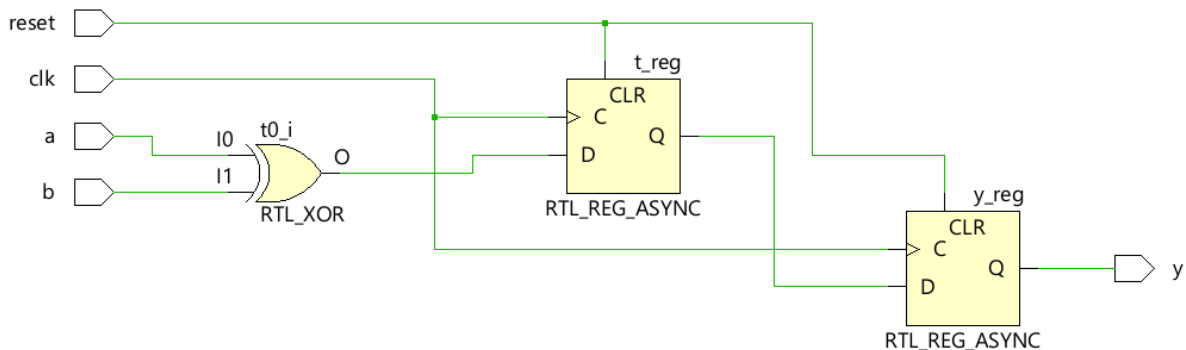
It must be design 1 as that design must remember the old value of t (since we are retrieving the value for t before it is computed).

Synthesise the two designs. Look at the RTL or technology view of the synthesised circuits and try to understand why they implement the respective VHDL designs correctly.

Design 1 is implemented correctly because there is an extra register, which saves the previous value of t , and it is this previous value that becomes y at the next clock edge



Design 2 is implemented correctly because there is no need to store the previous value of t .



What determines whether a signal/variable will be implemented as a register or not?

Whether a variable is stored in a register or not, is determined by if we need to remember the previous (last clock edge) value.

Can a signal or variable be written to in more than one process? Can they be read in more than one process?

Signals should be declared in the architecture and, be written only by one process to avoid conflicting drivers, and can read by multiple processes.

Variable should be declared inside the process, and is therefore only accessible to that process.

Task 3

Why is the process not synthesizable without 'b' in the sensitivity list?

The process can't synthesise without b in the sensitivity list, because y is dependant on changes in both a and b . (The code describes an **AND** operation between a and b)

Is there any reason to suspect that the generated circuit will not work like during simulation? Explain

We suspect that it won't work exactly like what was simulated. This is because when simulating, it runs without recognising b as an input, while the synthesis tool adds hardware to support it. Therefore it will behave differently.

If yes, what should you do in this situation to make sure the circuit conforms to simulation results?

In this case, we would need to add b to the sensitivity list.

Task 4

Problems with oscillations from combinational loops will typically be uncovered during simulation. Can you think of any situation where this is not the case?

If the value creating the loop is not present in the sensitivity list, it will not uncover the loop until it is synthesised.

In what situations is it permissible to have the same signal/variable on both sides of an assignment? (E.g., $b \leq a \text{ xor } b$;))

This is often used in sequential logic, but we cannot find a use case for combinational logic.